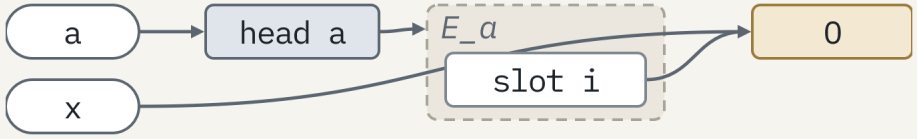
Read, then mutate through a variable

supported/ReadMutate.scala verifies

val x = a(i)

```
var x: Box = a(i)
```



---

x.value = 1

```
x = Box(1)
a = a.updated(i, x)
```

---

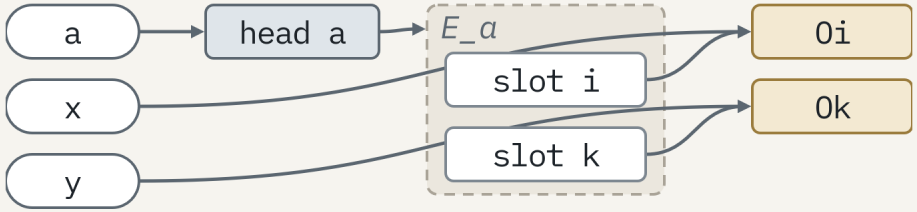
assert(a(i).value == 1)

Two slots that may be the same position

supported/TwoSlotsEqualIndex.scala verifies

val x = a(i)
val y = a(k)

```
var x: Box = a(i)
var y: Box = a(k)
```



---

x.value = 1

```
x = Box(1)
a = a.updated(i, x)
y = a(k)
```

---

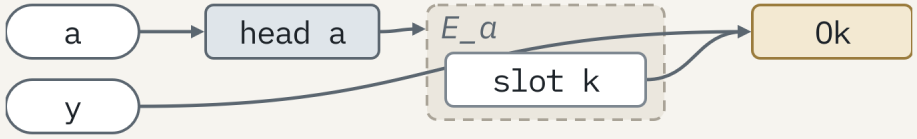
assert(i != k || y.value == 1)

Writing while another element is bound

supported/WriteWhileBoundDistinct.scala **verifies, with i != k in the require**

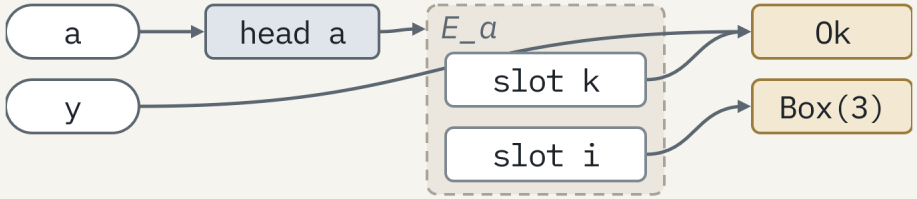
val y = a(k)

```
var y: Box = a(k)
```



a(i) = Box(3)

```
assert(i != k, "distinct collection indices")
a = a.updated(i, Box(3))
```



y.value = 1

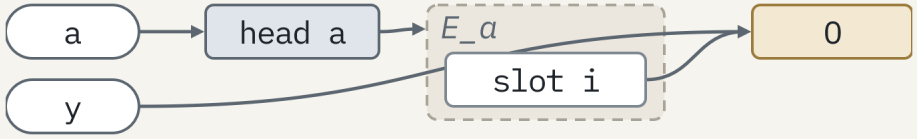
```
y = Box(1)
a = a.updated(k, y)
```

A write in one branch

supported/WriteInBranchWhileBound.scala **verifies**

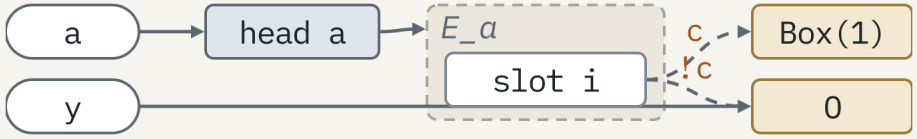
`val y = a(i)`

```
var y: Box = a(i)
```



`if (c) a(i) = Box(1)`

```
val cond: Boolean = c
if (cond) {
  a = a.updated(i, Box(1))
}
```



`y.value = 2`

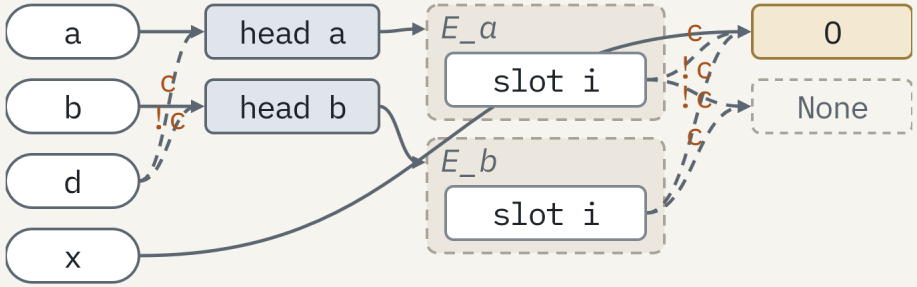
```
y = Box(2)
if (!c) {
  a = a.updated(i, y)
}
```

A read through a conditional collection

supported/ConditionalCollectionRead.scala verifies

val d = if (c) a else b
val x = d(i)

```
val cond: Boolean = c
val d: Array[Box] = if (cond) a else b
var x: Box = d(i)
```



---

x.value = 1

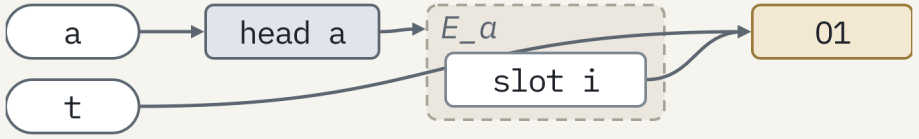
```
x = Box(1)
if (c) { a = a.updated(i, x) }
if (!c) { b = b.updated(i, x) }
```

Swap

supported/Swap.scala verifies

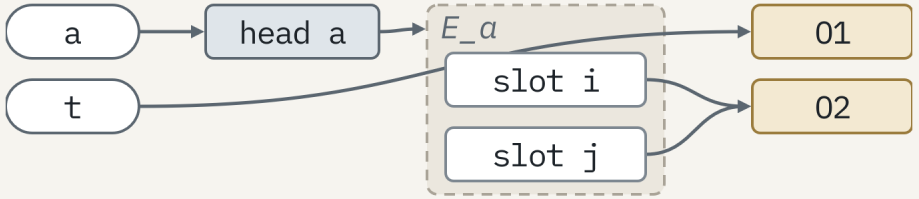
`val t = a(i)`

```
val t: Box = a(i)
```



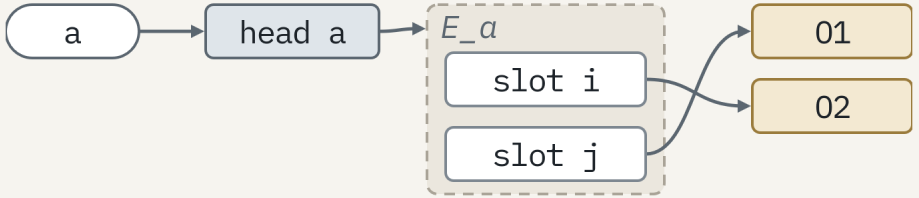
`a(i) = a(j)`

```
a = a.updated(i, a(j))
```



`a(j) = t`

```
a = a.updated(j, t)
```

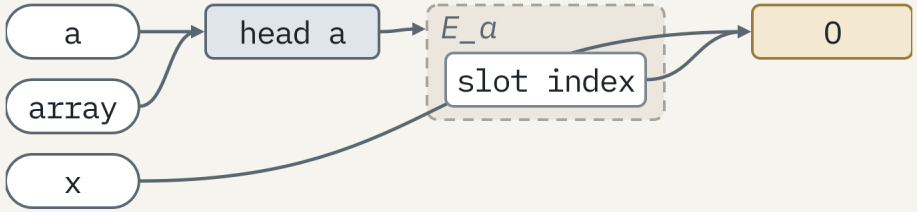


Indices that can change

supported/IndexChangesWhileBound.scala **verifies**

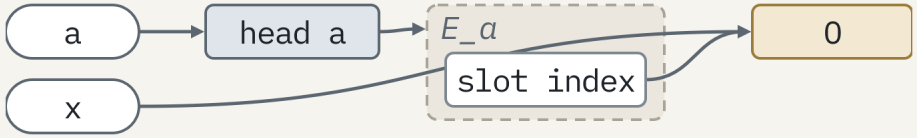
var i = 0
val x = a(i)

```
var i: Int = 0
val array: Array[Box] = a
val index: Int = i
var x: Box = array(index)
```



i = 1
x.value = 1

```
i = 1
x = Box(1)
a = a.updated(index, x)
```

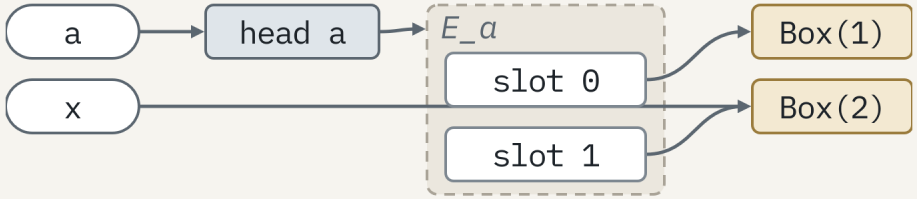


Copies

supported/UpdatedWhileBoundDistinct.scala **verifies**

var a = Array(Box(1), Box(2))
val x = a(1)

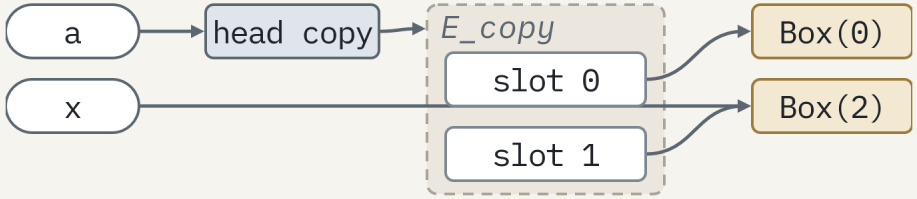
```
var a = Array(Box(1), Box(2))
var x: Box = a(1)
```



---

a = a.updated(0, Box(0))

```
a = {
  assert(0 != 1, "distinct collection indices")
  a.updated(0, Box(0))
}
```



---

x.value = 5

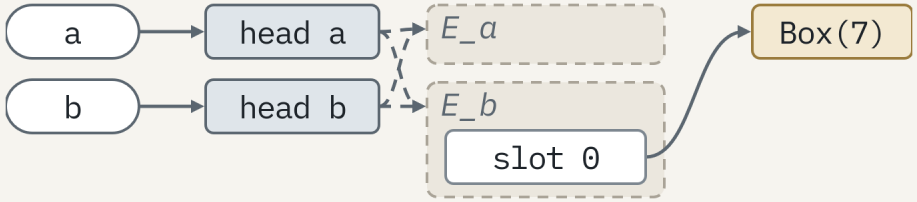
```
x = Box(5)
a = a.updated(1, x)
```

A copy whose source stays alive

unsupported/CopySourceReadAfter.scala **rejected**

```
val b = a.updated(0, Box(7))
```

```
 val b = a.updated(0, Box(7))
```



```
a(0).value = 5
```

```
a collection and its copy are both used after the copy -- they
share their mutable elements
```

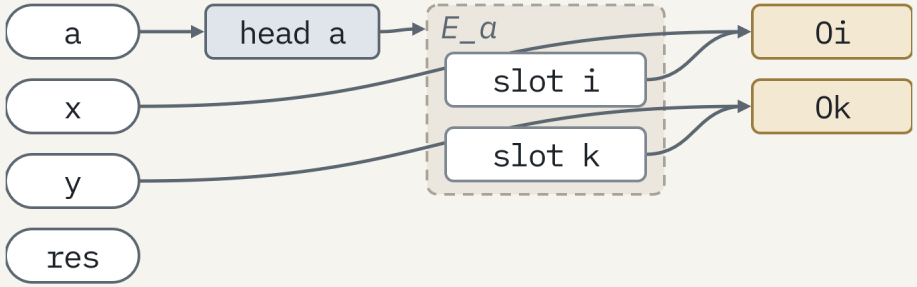
Calls

supported/CallDistinctIndices.scala **verifies, with i != k in the require**

```
val x = a(i)
val y = a(k)
g(x, y)
```

```
 var res = {
   assert(i != k, "distinct collection indices")
   g(x, y)
 }
```

```
x = res._1
res = (x, res._2, res._3)
a = a.updated(i, x)
y = a(k)
y = res._2
a = a.updated(k, y)
res = (res._1, y, res._3)
```

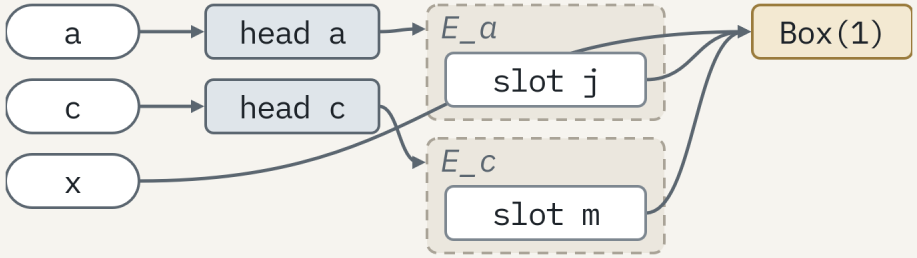


One object in two arrays

same_position/valid/SharedAcrossLocalArrays.scala verifies

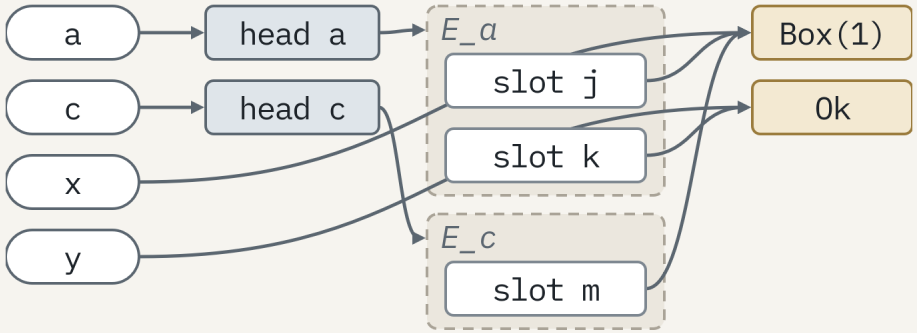
val x = Box(1)
a(j) = x
c(m) = x

```
var x: Box = Box(1)
a = a.updated(j, x)
c = c.updated(m, x)
```



val y = a(k)
y.value = 5

```
var y: Box = a(k)
y = Box(5)
a = a.updated(k, y)
c = c.updated(m, a(j))
x = a(j)
```

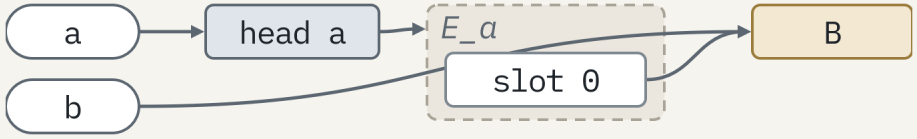


The function boundary

unsupported/NonFreshStore.scala **rejected**

```
def f(a: Array[Box], b: Box) = {
  a(0) = b
}
```

```
this function's result shares a mutable object with one of its
mutable parameters, or within itself -- results must be fresh
```



```
val t = a(i)
a(i) = a(j)
a(j) = t
```

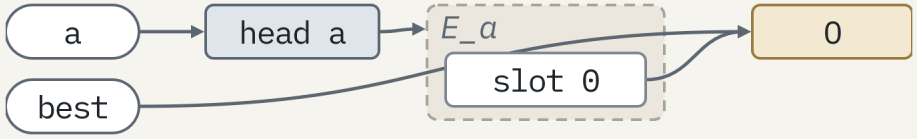
```
val t: Box = a(i)
a = a.updated(i, a(j))
a = a.updated(j, t)
```

A var bound to an element

supported/VarBoundToElement.scala **verifies**

var best = a(0)

```
var a: Array[Box] = a
var best: Box = a(0)
```



---

best.value = 0

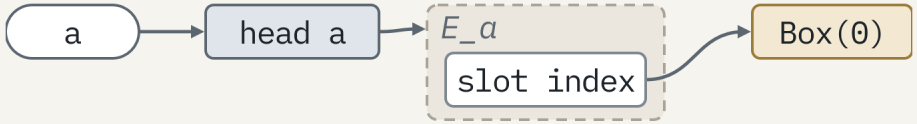
```
best = Box(0)
a = a.updated(0, best)
```

Loops

loops/invalid/LoopStoreOuterVariable.scala **second case rejected**

```
while (i < a.length) {
  a(i) = Box(0)
  i += 1
}
```

```
 while (i < a.length) {
val index: Int = i
   a = a.updated(index, Box(0))
   i = i + 1
 }
```



```
val x = Box(1)
while (...) {
  a(i) = x
  i += 1
}
```

an element written or read in this loop is still held by something else at the end of the iteration

